

Technical Document — Jagdeep Singh

1. API Strategy

As there are no official ways of getting data from Yahoo Finance or Google Finance. I had to find a work around as we needed

- **Yahoo Finance** — for CMP
- **Google Finance** — for P/E Ratio and Latest Earnings

So for —

- Yahoo Finance I found `yahoo-finance2` for CMP and fallback data
 - Google Finance I had to scrape data from there Quote pages using `Cheerio` for P/E ratio and latest earnings data
- The problem is Google Finance doesn't show all the data for all 26 stocks. For example, LTIMindtree's PE ratio just isn't there on its Google Finance page so I had to build a fallback chain instead of relying on one source.

- **Google Finance** (primary) - scraped P/E and EPS.
- **Yahoo trailing PE and EPS** (fallback) - if Google doesn't have it, I used `trailingPE` and `epsTrailingTwelveMonths` from the same Yahoo response. I already fetched CMP so this doesn't cost any extra API calls.
- **N/A** (last resort) - If neither source has the value (like a stock with negative EPS, which genuinely has no valid PE), I show N/A instead of making something.

For LTI Mindtree Google gives nothing but Yahoo's trailingPE kicks in and shows ~24.25 instead of a blank.

2. Rate Limiting

Hitting two scraped unofficial sources for 26 stocks on every request was risky so I did two things to avoid getting rate-limited or blocked:

- **Batching** — Instead of firing all 26 requests at once, I split them into batches of 5 for quotes and 6 for fundamentals using Promise so it's parallel but not all at once.
- **Caching based on market hours** — I build a simple in-memory cache that holds quotes for 15 seconds and fundamentals for 5 minutes while the market is open, since PE and EPS don't really change during the day anyway. When the market is closed both get cached for 6 hours instead. The dashboard isn't scraping data at 2 am for no reason. It just loads the last known value instantly.

The market hours are calculated on the server using Luxon, checking against Indian Standard Time directly from Monday till Friday, from 9 am to 3:30 pm, so it doesn't depend on whatever time zone the user's browser thinks it is.

3. Ticker mapping

Yahoo and Google don't use the same ticker format for the same stock, which caused some issues early on.

- Normally Yahoo uses `.NS` or `.BO` and Google uses `:NSE` or `:BOM`. I wrote a small converter function to handle this pattern.
- But some stocks don't follow the pattern at all. LTI Mindtree is `LTIM.NS` on Yahoo but `LTM:NSE` on Google, and Savani Financials only trades on BSE (`511577.BO`) so no NSE listing

For these exceptions I just made override maps: one for Yahoo tickers, one for Google tickers. Instead of writing special case logic inside the fetch function. That way if another mismatched stock shows up later, it's just a one-line addition instead of touching the actual fetch code.

4. Data accuracy

Since the Google Finance data comes from scraping and not an actual API, it depends on a specific CSS class name for their quote page. If Google changes their page layout, the scraper could break for every stock at once. This is the real risk and not something I can fully solve, only reduce.

The Yahoo fallback helps here even if the scraper breaks completely. Yahoo's trailing PE and EPS still show up instead of the dashboard going blank. I haven't built anything to detect when the scraper just broke for everyone. That would be a good next step but wasn't in scope for this.

Where a value is genuinely missing (like negative EPS having no real PE), I show N/A on purpose instead of guessing a number since that's more honest than displaying something misleading.

5. Asynchronous operations

All the API calls are async and run in parallel within each batch using Promise. With 26 stocks, doing this one by one would make the dashboard noticeably slow to load.

On the frontend I use a polling setup instead of one long request. I call `/api/quotes` every 15 seconds but only while the market is actually open. There's a separate check running every 15 seconds that watches whether the market just opened or closed and starts and stops the poll accordingly so it doesn't keep polling uselessly after hours and the moment the market reopens it refetches immediately instead of waiting for the next 15-seconds.

6. Error handling

Each stock's fetch (both quote and fundamentals) is wrapped in its own try/catch. So, if one ticker fails — timeout bad response, missing data on the page — It doesn't take down the rest. A failed fetch just returns null with an error field instead of throwing and the UI shows N/A or loading for the specific stock instead of crashing.

On top of that I track if the whole `/api/quotes` or `/api/fundamentals` call itself fails and show a small retry banner on the dashboard (so the user knows the data might be stale or can manually refresh). There's also a React error boundary wrapping the table so even an unexpected render bug in one row won't take down the whole page.

7. Performance optimisation

Two main things help here:

- **Caching** — The in-memory TTL (Time to Live) cache in front of both quote and fundamentals routes is the main thing keeping this fast and within rate limits, adjusting itself based on market, as I mentioned above.
- **Selective pooling** — The frontend only keeps pulling quotes while the market is open and fundamentals aren't refreshed on every single quote pull since P/E and EPS don't change that quickly.
- **Memoization** — I used `useMemo` for the table row data and column definitions so they don't get rebuilt on every render, and used `useCallback` for the fetch function so the polling interval and effects don't end up with stale closure.

8. Functionality, code quality, and UI

Functionality — At a high level the dashboard shows all 26 holdings grouped into six sectors, with each row showing:

- Purchase price
- Quantity
- Investment
- Portfolio %
- CMP
- Present value
- Gain/Loss
- PE ratio
- Latest earnings

CMP updates every 15 seconds while the market is open and gain/loss is colour-coded green or red so it's easy to scan at a glance. Sector-level totals investment, present value, and gain/loss sit alongside the table so the portfolio can be read at both the stock level and sector level.

Code quality — The whole project is TypeScript end-to-end, with shared types used across the API routes and the frontend. Logic is split by concern:

- Scraping and market hours logic lives in `lib/`.
- Data fetching lives in `app/api/`.
- Rendering lives in `components/`.

Each piece can be read and changed on its own without touching the others. Ticker overrides and formats are pulled out into their own small files instead of being inline, which is why adding a new mismatch ticker later is a one-line change rather than a rewrite.

User interface — The dashboard uses a two-panel layout on desktop (a sticky summary sidebar plus the main table) and switches to a drawer-based layout on mobile so the summary doesn't eat up screen space on a small screen. The table itself keeps its header and the stock column pinned while scrolling so you don't lose track of which row you are looking at. On a long list sector filters let you filter the table down to one sector at a time and a small badge shows whether the market is currently open or closed at a glance.

9. Limitations

- **Scraper fragility** — Google Finance's page structure isn't guaranteed to stay the same so a layout change on their end would break the scraper and would need a CSS selector update in the code.
- **In-memory cache only** — The cache lives in server memory and resets if the server restarts. For a real production setup with multiple instances, this would need something shared like Redis.

For locally running the repo, once we run `npm run dev`, the app may take some time to load all the data into the cache but once the cache is loaded, the app is snappy and very responsive.

Github Repo Link : <https://github.com/jagdep-singh/octa-byte>

Deployment Link : <https://octa-byte-seven.vercel.app>